UNIT III: Raspberry Pi with Python Programming for IoT

Syllabus

- Introduction to Python Programming Basics
- Overview of Raspberry Pi and Its Role in IoT
- Interfacing Raspberry Pi with Sensors and Actuators (UART, SPI, I2C)
- Data Acquisition and Local Processing on Raspberry Pi
- Sending Data to Cloud Platforms
- Implementation of IoT Projects Using Raspberry Pi
- Case Studies: Smart Home Automation, Healthcare Monitoring, Environmental Sensing

1. Programming for IoT

1.1 What is IoT programming?

An IoT system has four layers: sensing devices, connectivity, data processing, and application/user interface. Programming for IoT means writing software that reads data from sensors, processes it, controls actuators, and communicates over a network. Code runs at different places: on the **device (edge)**, on a **gateway** (such as a Raspberry Pi), and in the **cloud**.

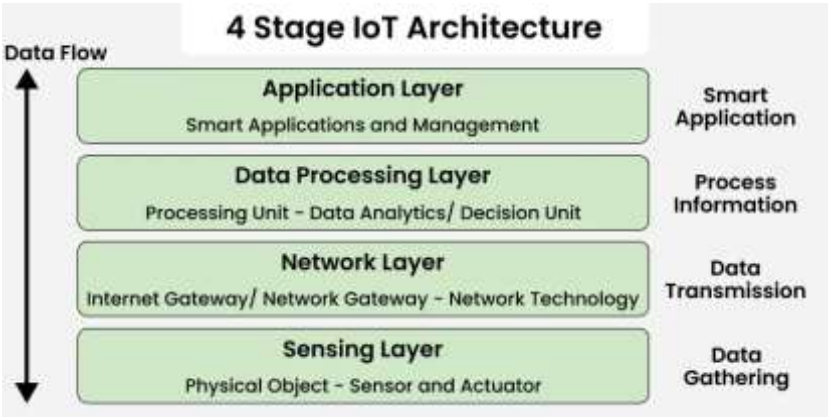


Fig 1.1: IoT Architecture (Four-Layer Model)

1.2 Choosing a language

Language	Where used	Remarks
Python	Raspberry Pi, gateways, cloud scripts, ML	Easy syntax, huge library support (GPIO, MQTT, requests)
C / C++	Microcontrollers (Arduino, ESP32), firmware	Fast, memory-efficient, low-level hardware control
JavaScript (Node.js)	Web dashboards, Node-RED, cloud functions	Event-driven; good for real-time web apps
Java	Android apps, enterprise back-ends	Portable, strong ecosystem

1.3 Why Python for IoT?

- Simple, readable syntax so prototypes are built quickly.
- Rich libraries: **RPi.GPIO, gpiozero, spidev, smbus2, pyserial, paho-mqtt, requests, Flask, NumPy, pandas**.
- Runs on Linux (Raspberry Pi OS), so networking, files, and multitasking are available.
- Large community and cross-platform support; MicroPython extends it to microcontrollers.
- Good for data analysis and machine learning at the edge.

1.4 General programming flow of an IoT application

1. Initialize hardware (GPIO pins, buses) and network.
2. Read sensor data (polling or interrupt based).
3. Process/validate data (filter, convert units, threshold check).
4. Take local action (control actuator) if required.
5. Transmit data to cloud (MQTT/HTTP).
6. Sleep or wait, then repeat in a loop.

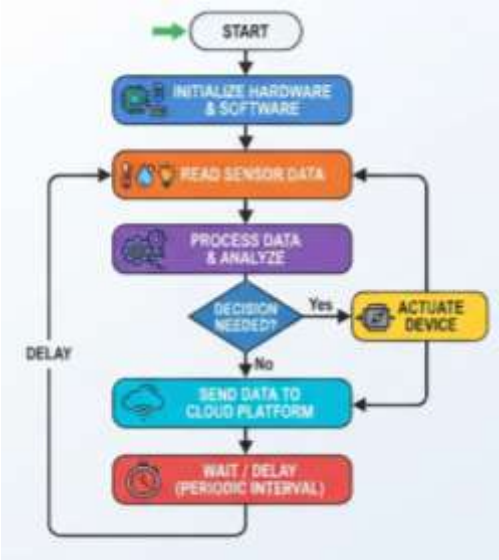


Fig 1.2: General Programming Flow of an IoT Application (Flowchart)

2. Introduction to Python Programming Basics

2.1 Features of Python

- Interpreted, high-level, and dynamically typed.
- Object-oriented as well as procedural.
- Indentation defines code blocks (no braces).
- Free and open source with extensive standard library.

2.2 Variables and data types

A variable is a name referring to a value; no type declaration is needed. Common types: **int, float, str, bool, list, tuple, dict, set**.

Type	Example	Property
int / float	count = 5; temp = 28.6	Numeric values
str	name = "Pi"	Immutable text
list	[10, 20, 30]	Ordered, mutable
tuple	(10, 20)	Ordered, immutable
dict	{"temp": 28, "hum": 60}	Key-value pairs (like JSON)
bool	True / False	Logical values

2.3 Operators

- Arithmetic: + - * / // % **
- Relational: == != < > <= >=
- Logical: and, or, not
- Assignment: = += -= etc.

2.4 Control statements

```
temp = 32
if temp > 35:
    print("High")
elif temp > 25:
    print("Normal")
else:
    print("Low")

for i in range(5):      # loop 5 times
    print(i)

while True:             # infinite loop (typical in IoT)
    pass
```

2.5 Functions

```
def c_to_f(c):
    return (c * 9/5) + 32

print(c_to_f(30))   # 86.0
```

2.6 Modules and packages

A module is a .py file; import it to reuse code. Install external packages with pip.

```
import time
from gpiozero import LED
# pip3 install paho-mqtt
```

2.7 Exception handling and file handling

```
try:
    value = read_sensor()
except Exception as e:
    print("Error:", e)
finally:
    cleanup()

with open("log.csv", "a") as f:
    f.write("28.5,60\n")
```

2.8 Other useful concepts for IoT

- **Classes/objects** to model devices (Sensor, Actuator).
- **Threading** to run sensing and uploading together.
- **JSON** (json module) for structured data exchange.
- **datetime/time** for timestamps and delays.

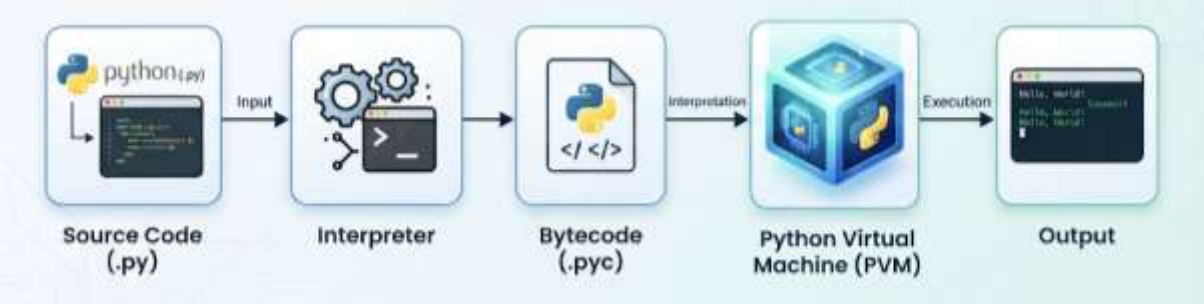


Fig 2.1: Python Program Execution Flow

3. Overview of Raspberry Pi and Its Role in IoT

3.1 Introduction

Raspberry Pi is a low-cost, credit-card-sized single-board computer (SBC) developed by the Raspberry Pi Foundation (UK). It runs a full Linux-based OS (Raspberry Pi OS) and has GPIO pins to connect with the physical world, which makes it ideal as an IoT node and gateway.

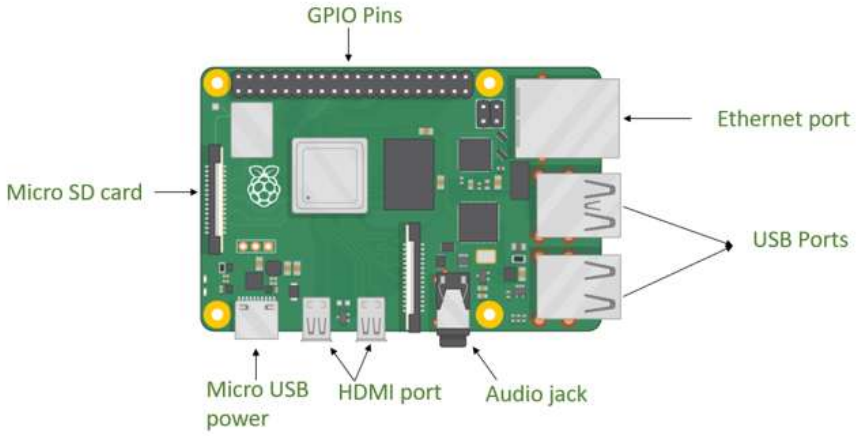


Fig 3.1: Raspberry Pi Board with Labelled Components

3.2 Key specifications (Raspberry Pi 4 Model B)

Feature	Specification
Processor	Broadcom BCM2711, quad-core Cortex-A72 (ARM v8) 64-bit, 1.5 GHz
RAM	1/2/4/8 GB LPDDR4
Connectivity	2.4/5 GHz Wi-Fi, Bluetooth 5.0, Gigabit Ethernet
USB	2 x USB 3.0, 2 x USB 2.0
Video	2 x micro-HDMI (up to 4K)
GPIO	40-pin header (26 usable GPIO), 3.3 V logic
Storage	microSD card
Power	5 V, 3 A via USB-C

3.3 Raspberry Pi GPIO

- 40-pin header: power pins (3.3 V, 5 V), ground pins, and GPIO pins.
- Logic level is **3.3 V**; applying 5 V to a GPIO pin can damage the board.
- Special-function pins: I2C (GPIO2 SDA, GPIO3 SCL), SPI (GPIO10 MOSI, GPIO9 MISO, GPIO11 SCLK, GPIO8 CE0), UART (GPIO14 TXD, GPIO15 RXD), PWM.
- Two numbering schemes: BCM (GPIO number) and BOARD (physical pin number).

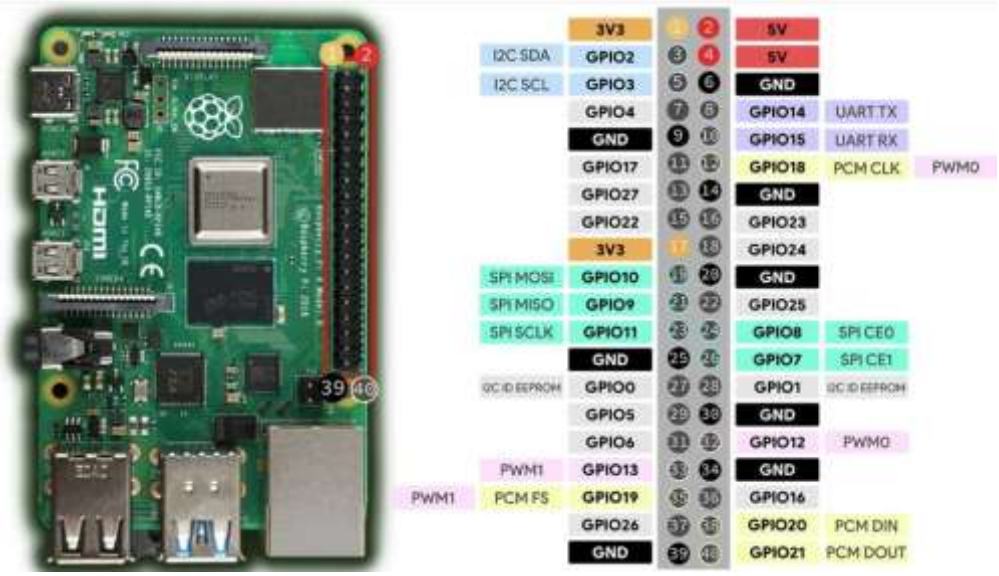


Fig 3.2: Raspberry Pi 40-Pin GPIO Pinout Diagram

3.4 Operating system and setup

1. Download Raspberry Pi Imager and write Raspberry Pi OS to a microSD card.
2. Insert the card, connect power, keyboard/monitor (or enable SSH headless).
3. Configure Wi-Fi, enable interfaces using sudo raspi-config (I2C, SPI, Serial, SSH).
4. Update packages: sudo apt update && sudo apt upgrade.
5. Write and run Python programs: python3 file.py.

3.5 Role of Raspberry Pi in IoT

- **IoT node:** directly interfaces sensors and actuators.
- **Gateway/edge device:** collects data from many low-power devices (BLE, Zigbee, LoRa), pre-processes it and forwards it to the cloud.
- **Local server:** hosts MQTT broker (Mosquitto), web server (Flask), database (SQLite) and dashboards (Node-RED).
- **Edge intelligence:** runs ML models (TensorFlow Lite, OpenCV) locally to reduce latency and bandwidth.
- **Prototyping platform:** fast development for academic and industrial proof-of-concept.

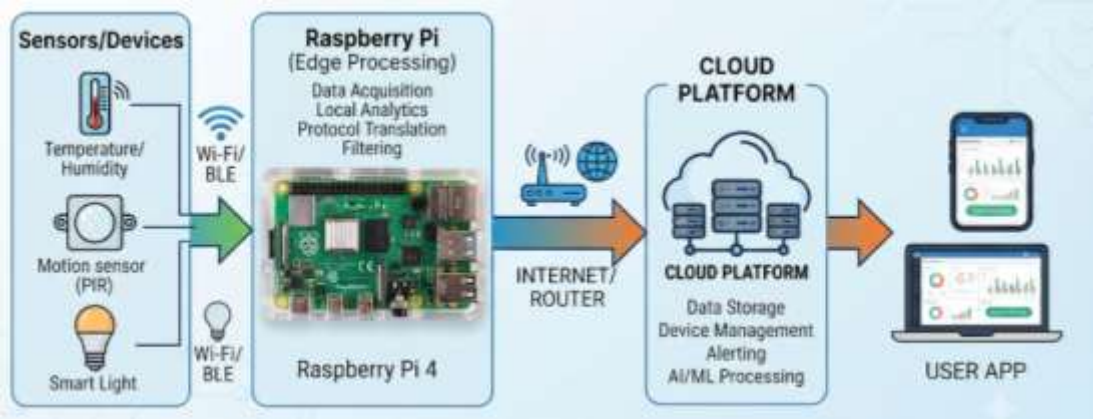


Fig 3.3: Role of Raspberry Pi as IoT Gateway

3.6 Raspberry Pi vs Arduino

Parameter	Raspberry Pi	Arduino
Type	Single-board computer (microprocessor)	Microcontroller board
OS	Linux-based	No OS (bare metal)
Speed / Memory	GHz range, GBs RAM	16 MHz, KBs of RAM
Analog input	None (needs ADC like MCP3008)	Built-in ADC
Networking	Built-in Wi-Fi, Ethernet	Needs shield/module (except some boards)
Languages	Python, C, Java, etc.	C/C++ (Arduino IDE)
Best for	Gateways, multimedia, ML, servers	Simple real-time sensor control

4. Interfacing Raspberry Pi with Sensors and Actuators

4.1 Sensors and actuators

A **sensor** converts a physical quantity (temperature, light, motion) into an electrical signal. An **actuator** converts an electrical signal into physical action (LED, relay, motor, buzzer). Raspberry Pi connects to them either through simple GPIO or through communication protocols (UART, SPI, I2C).

Category	Examples	Interface
Digital sensors	PIR motion, IR obstacle, door switch	GPIO input

Category	Examples	Interface
Digital-protocol sensors	DHT11/22 (single wire), DS18B20 (1-Wire), BMP280, MAX30100	GPIO / I2C
Analog sensors	LDR, MQ-135 gas, soil moisture, LM35	ADC (MCP3008 via SPI, ADS1115 via I2C)
Actuators	LED, buzzer, relay, DC/servo motor	GPIO output / PWM

4.2 GPIO interfacing

Example 1: LED blink (output)

```
from gpiozero import LED
from time import sleep

led = LED(17)          # BCM pin 17
while True:
    led.on();  sleep(1)
    led.off(); sleep(1)
```

Example 2: PIR motion sensor (input)

```
from gpiozero import MotionSensor, Buzzer
pir = MotionSensor(4)
buzzer = Buzzer(27)
while True:
    pir.wait_for_motion()
    buzzer.on()
    pir.wait_for_no_motion()
    buzzer.off()
```

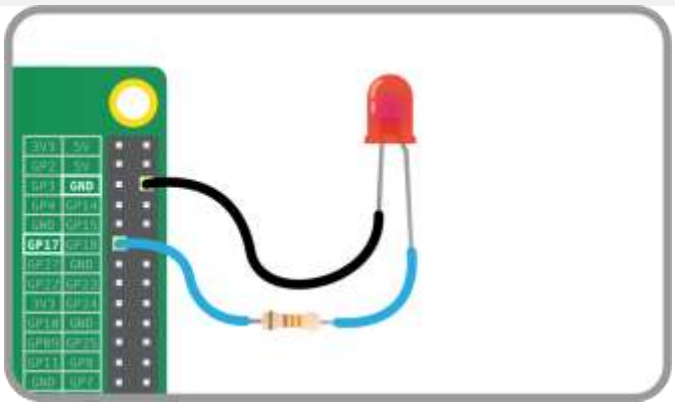


Fig 4.1: LED Interfacing with Raspberry Pi (Circuit Diagram)

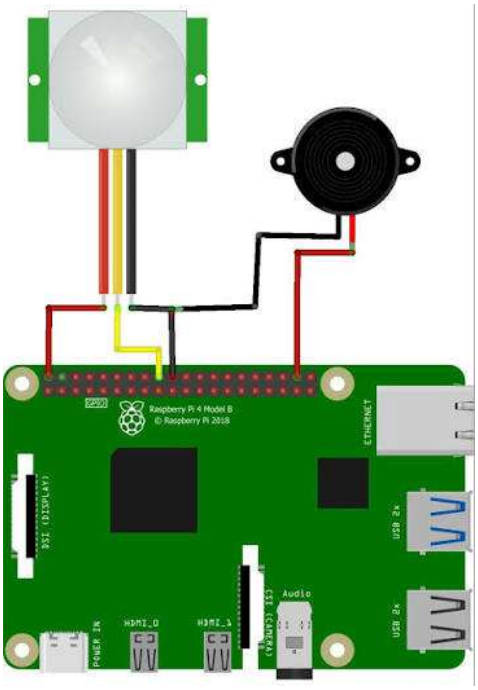


Fig 4.2: PIR Sensor and Buzzer Connection with Raspberry Pi

4.3 UART (Universal Asynchronous Receiver/Transmitter)

- **Asynchronous serial** communication: no shared clock; both sides agree on baud rate (e.g., 9600).
- Two data lines: **TX** (transmit) and **RX** (receive), plus common GND. TX of one device connects to RX of the other (cross connection).
- Frame format: start bit, 5-9 data bits, optional parity bit, stop bit(s).
- Point-to-point (one device to one device); full duplex.
- Pi pins: GPIO14 (TXD), GPIO15 (RXD). Enable serial using raspi-config; device file /dev/serial0.
- Used for GPS modules (NEO-6M), GSM modules (SIM800), Bluetooth HC-05, fingerprint sensors, Arduino communication.

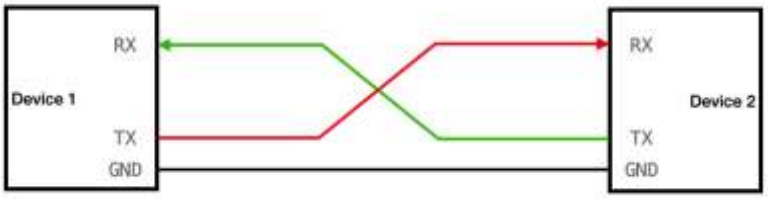


Fig 4.3: UART Communication Between Two Devices

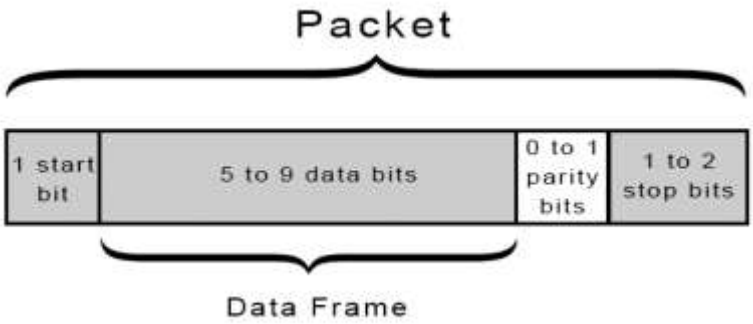


Fig 4.4: UART Data Frame Format

```
import serial
ser = serial.Serial('/dev/serial0', baudrate=9600, timeout=1)
ser.write(b'Hello\n')            # send
data = ser.readline().decode().strip()   # receive
print(data)
ser.close()
```

4.4 SPI (Serial Peripheral Interface)

- **Synchronous**, full-duplex, master-slave serial bus; high speed (several MHz).
- Four lines: **MOSI** (master out slave in), **MISO** (master in slave out), **SCLK** (clock from master), **CS/SS** (chip select, one per slave).
- Pi is the master; Pi pins: MOSI GPIO10, MISO GPIO9, SCLK GPIO11, CE0 GPIO8, CE1 GPIO7.
- Master pulls CS low to select a slave; data is shifted out and in simultaneously on each clock pulse.
- Advantages: fast, simple, no addressing. Disadvantages: more wires, no acknowledgement, number of CS lines grows with slaves.
- Used for ADC (MCP3008), displays (TFT, OLED), SD cards, RFID (MFRC522), LoRa modules.

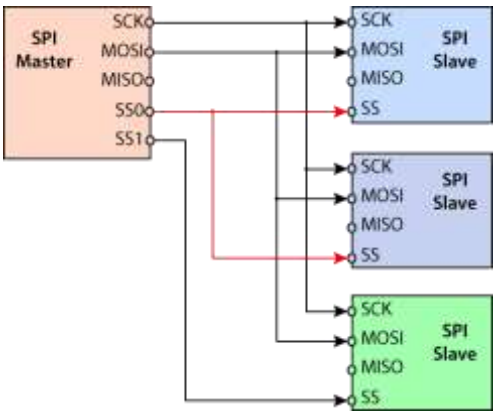


Fig 4.5: SPI Master-Slave Connection (Single Master, Multiple Slaves)

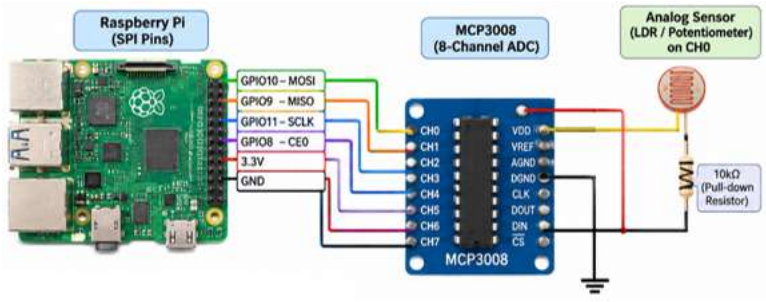


Fig 4.6: Interfacing MCP3008 ADC with Raspberry Pi over SPI

```
import spidev
spi = spidev.SpiDev()
spi.open(0, 0)               # bus 0, device CE0
spi.max_speed_hz = 1350000

def read_adc(ch):
```

```
    r = spi.xfer2([1, (8 + ch) << 4, 0])
    return ((r[1] & 3) << 8) + r[2]   # 10-bit value

v = read_adc(0)
print("Voltage:", v * 3.3 / 1023)
```

4.5 I2C (Inter-Integrated Circuit)

- **Synchronous**, half-duplex, multi-master / multi-slave serial bus using only **two wires**: **SDA** (data) and **SCL** (clock).
- Each slave has a unique **7-bit address** (e.g., BMP280 = 0x76, MPU6050 = 0x68, OLED = 0x3C).
- Lines are open-drain and need **pull-up resistors** (Raspberry Pi has built-in 1.8 kOhm on GPIO2 and GPIO3).
- Communication: START condition, address + R/W bit, ACK, data bytes with ACK, STOP condition.
- Standard speed 100 kbps, fast mode 400 kbps.
- Pi pins: GPIO2 (SDA), GPIO3 (SCL). Enable I2C in raspi-config and check devices with i2cdetect -y 1.
- Used for BMP280/BME280, MPU6050, RTC DS3231, ADS1115, OLED, MAX30100.

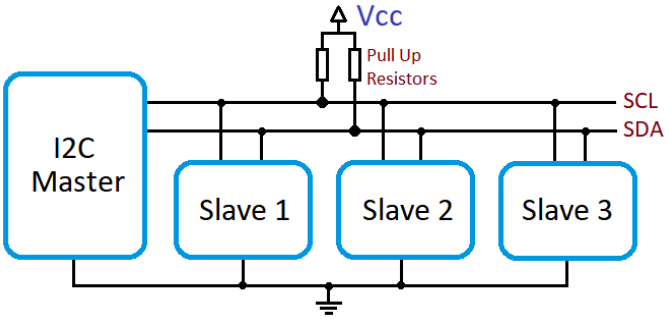


Fig 4.7: I2C Bus with One Master and Multiple Slaves

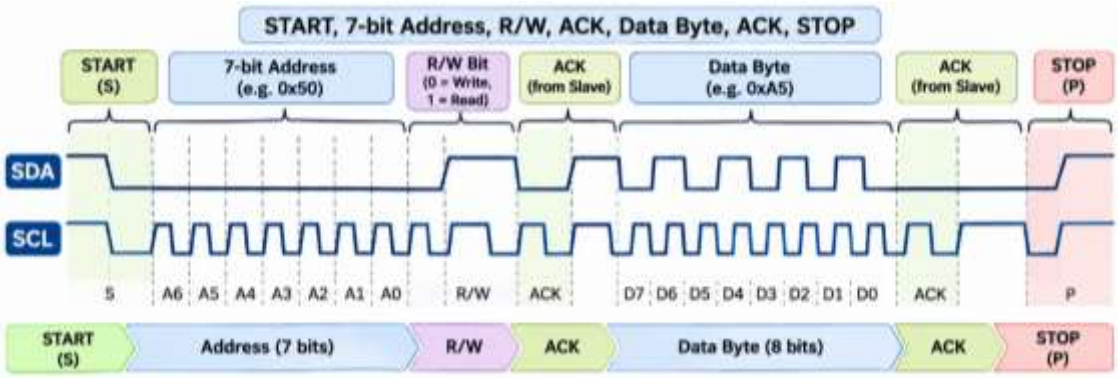


Fig 4.8: I2C Communication Frame (Timing Diagram)

```
import smbus2, time
bus = smbus2.SMBus(1)         # I2C bus 1
ADDR = 0x48                  # e.g., ADS1115 / sensor address
data = bus.read_i2c_block_data(ADDR, 0x00, 2)
print(data)
```

4.6 Comparison of UART, SPI and I2C

Feature	UART	SPI	I2C
Clock	Asynchronous	Synchronous	Synchronous
Wires	2 (TX, RX)	4 + 1 per extra slave	2 (SDA, SCL)
Topology	Point-to-point	Single master, multi-slave	Multi-master, multi-slave
Speed	Up to ~1 Mbps	Highest (10+ Mbps)	100 kHz / 400 kHz / 3.4 MHz
Addressing	None	Chip select	7/10-bit address
Duplex	Full	Full	Half
Typical use	GPS, GSM, Bluetooth	ADC, display, SD card	Sensor ICs, RTC, OLED

4.7 Controlling actuators

- **Relay module:** lets low-voltage GPIO switch high-voltage AC loads (fan, bulb). Needs isolation/optocoupler.
- **Servo motor:** controlled by PWM signal (pulse width 1-2 ms).
- **DC motor:** needs motor driver (L293D/L298N) as GPIO cannot supply enough current.
- **Buzzer/LED:** direct GPIO with a series resistor.

```
from gpiozero import Servo, OutputDevice
relay = OutputDevice(23, active_high=False)
relay.on()      # switch ON appliance
servo = Servo(18)
servo.min(); servo.mid(); servo.max()
```

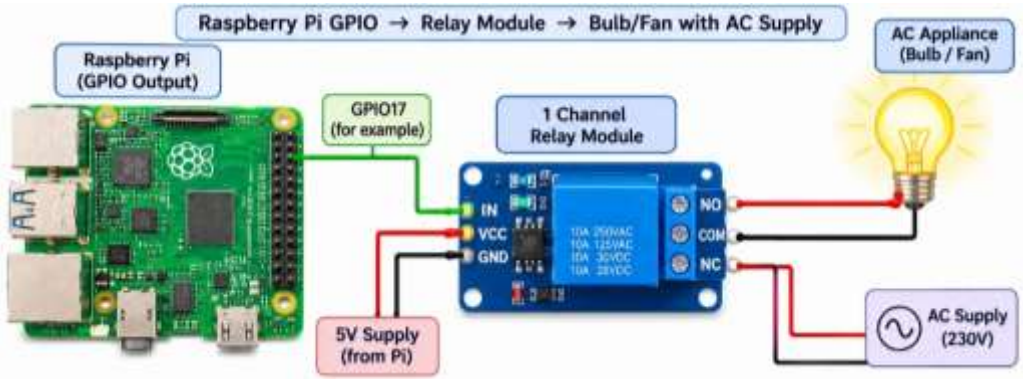


Fig 4.9: Relay Module Interfacing for Controlling an AC Appliance

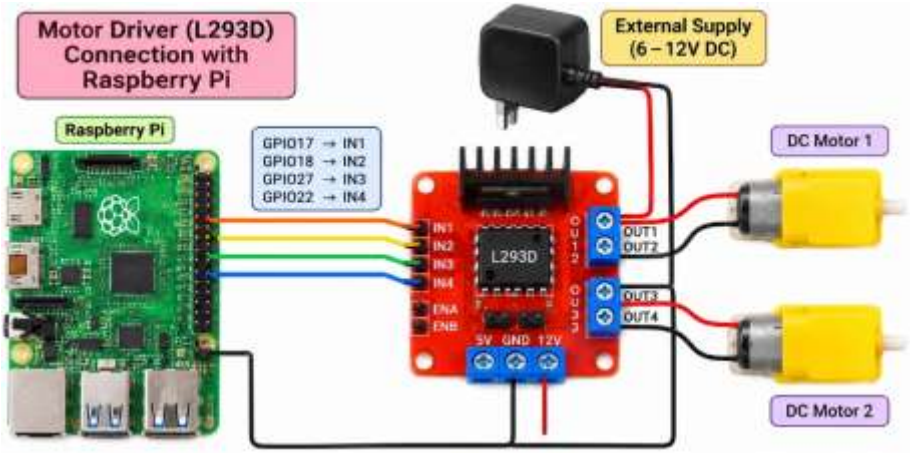


Fig 4.10: Motor Driver (L293D) Connection with Raspberry Pi

5. Data Acquisition and Local Processing on Raspberry Pi

5.1 Data acquisition (DAQ)

Data acquisition is the process of sampling real-world signals and converting them into digital values that the computer can store and process. Stages: **sensing -> signal conditioning -> ADC -> processing -> storage/transmission**.

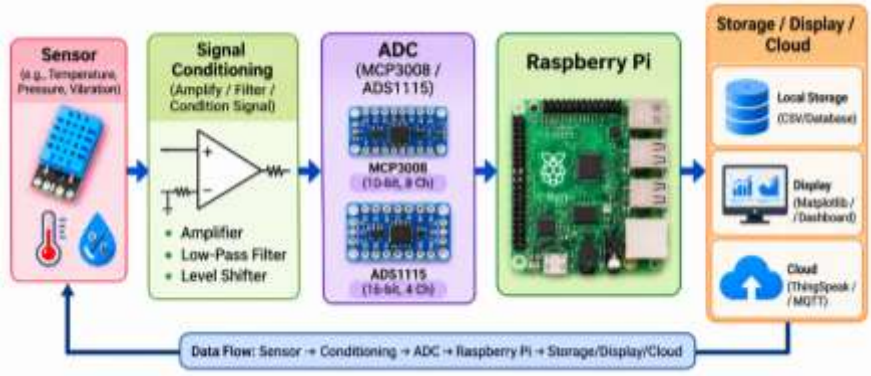


Fig 5.1: Data Acquisition Chain on Raspberry Pi

- **Sampling rate:** number of readings per second; must suit how quickly the quantity changes (Nyquist criterion for fast signals).
- **Resolution:** ADC bits (MCP3008 = 10-bit, ADS1115 = 16-bit).
- **Polling vs interrupt:** polling reads repeatedly in a loop; interrupts (edge detection) react only when an event occurs, saving CPU.
- Raspberry Pi has **no built-in ADC**, so analog sensors need an external ADC.

5.2 Example: reading DHT22 and storing locally

```
import adafruit_dht, board, time, csv, datetime
dht = adafruit_dht.DHT22(board.D4)

while True:
    try:
        t = dht.temperature
        h = dht.humidity
        ts = datetime.datetime.now().isoformat()
        with open('data.csv', 'a', newline='') as f:
```

```
            csv.writer(f).writerow([ts, t, h])
        print(ts, t, h)
    except RuntimeError:
        pass                # DHT occasionally fails; retry
    time.sleep(5)
```

5.3 Local (edge) processing

Instead of sending all raw data to the cloud, the Pi processes data locally. Benefits: lower latency, reduced bandwidth and cloud cost, offline operation, better privacy, and faster local decisions.

- **Filtering/Noise removal:** moving average, median filter.
- **Calibration/Unit conversion:** raw ADC value to degrees/lux/ppm.
- **Validation:** discard out-of-range or missing readings.
- **Aggregation:** send hourly average/min/max instead of every sample.
- **Threshold/rule-based actions:** if temperature > 40, switch ON fan immediately.
- **Analytics/ML:** NumPy, pandas, OpenCV, TensorFlow Lite for anomaly detection and recognition.

```
from collections import deque
window = deque(maxlen=5)

def smooth(x):
    window.append(x)
    return sum(window) / len(window)

if smooth(temp) > 40:
    fan.on()
```

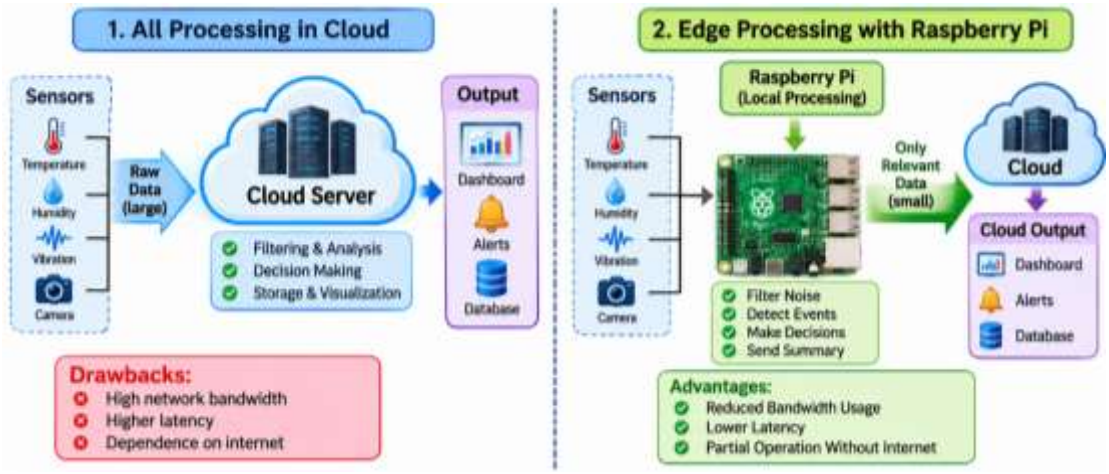


Fig 5.2: Edge vs Cloud Processing Architecture

5.4 Local storage options

Storage	Use
CSV / text files	Simple logging, easy to import into Excel/pandas
SQLite database	Lightweight SQL database for structured local data (import sqlite3)
JSON files	Configuration and structured records
InfluxDB / RRDtool	Time-series data on the Pi

5.5 Local visualization

- Console output, OLED/LCD display, matplotlib plots.
- Flask web page or Node-RED dashboard hosted on the Pi.

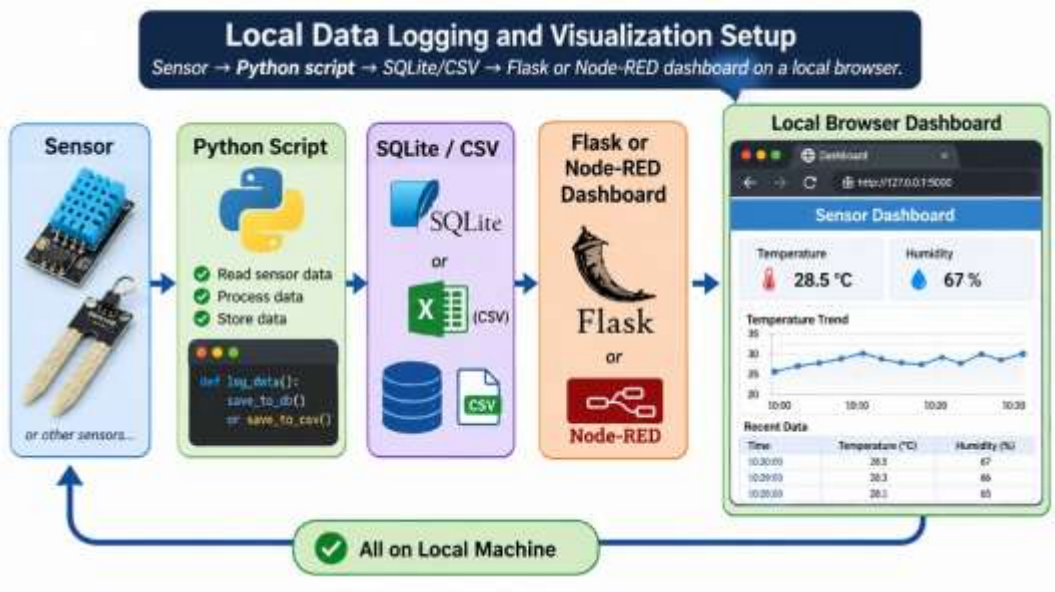


Fig 5.3: Local Data Logging and Visualization Setup

6. Sending Data to Cloud Platforms

6.1 Why the cloud?

- Remote monitoring and control from anywhere.
- Scalable storage and analytics, dashboards, alerts.
- Integration with mobile apps and other services.

6.2 IoT communication protocols

Protocol	Description	Typical use
HTTP/HTTPS (REST)	Request-response over TCP; heavier headers	Sending data using POST to ThingSpeak, Firebase, custom APIs
MQTT	Lightweight publish/subscribe via a broker over TCP; QoS 0,1,2; default port 1883 (8883 TLS)	AWS IoT Core, Azure IoT Hub, ThingsBoard, Adafruit IO
CoAP	Lightweight REST-like protocol over UDP	Constrained devices
WebSocket	Persistent two-way connection	Real-time dashboards

6.3 MQTT in detail

- **Broker:** central server routing messages (Mosquitto, cloud broker).
- **Publisher:** device sending messages to a **topic** (e.g., home/livingroom/temp).
- **Subscriber:** client subscribing to topics to receive messages.

- Features: small header, low power, **QoS levels**, retained messages, Last Will and Testament.

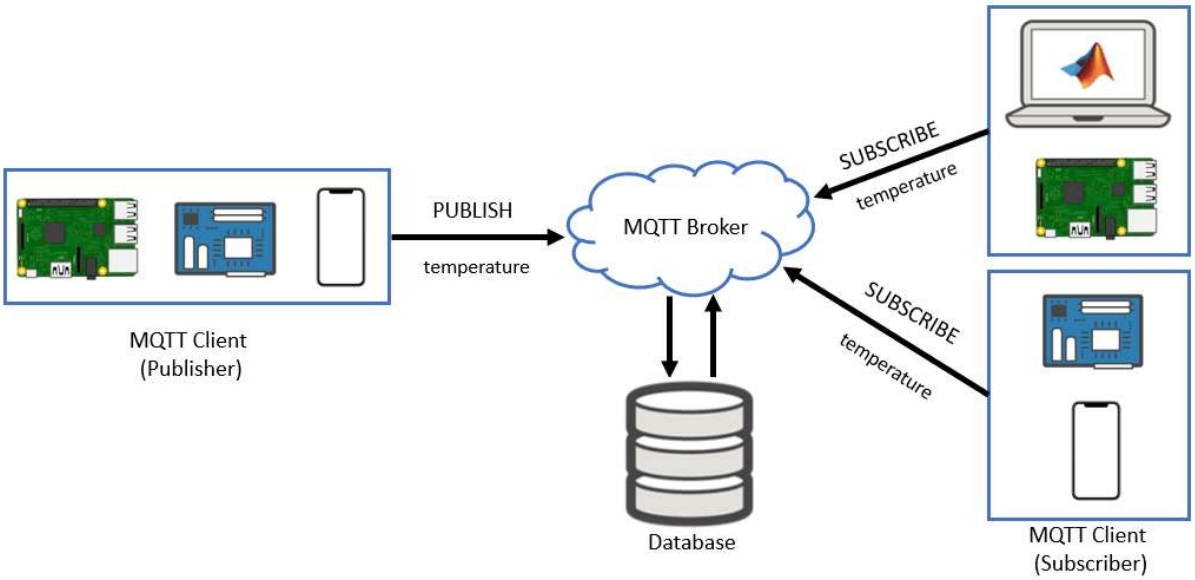


Fig 6.1: MQTT Publish/Subscribe Architecture

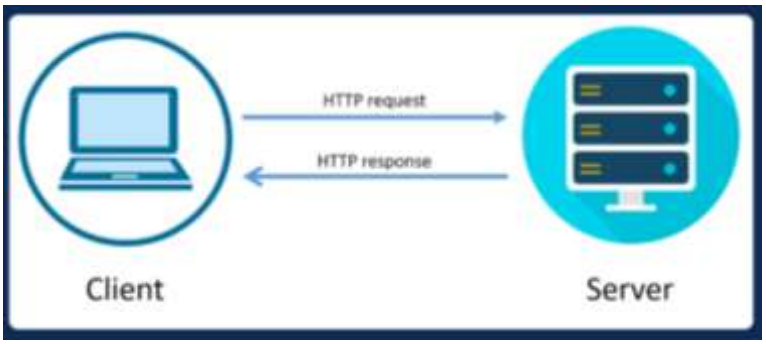


Fig 6.2: HTTP Request-Response Model

6.4 Popular cloud platforms

Platform	Features
ThingSpeak (MathWorks)	Free channels, HTTP/MQTT, built-in charts and MATLAB analytics; ideal for beginners
AWS IoT Core	Secure device connectivity via MQTT with certificates, rules engine, integration with S3, Lambda, DynamoDB
Microsoft Azure IoT Hub	Device management, bi-directional messaging, Stream Analytics
Google Cloud / Firebase	Realtime database, Firestore, cloud functions
Adafruit IO / Blynk / ThingsBoard	Easy dashboards, widgets and mobile apps

6.5 Example 1: Sending data to ThingSpeak (HTTP)

```
import requests, time
API_KEY = "YOUR_WRITE_API_KEY"
URL = "https://api.thingspeak.com/update"

while True:
    temp, hum = read_dht()
    r = requests.get(URL, params={'api_key': API_KEY,
                                  'field1': temp, 'field2': hum})
    print("Status:", r.status_code, "Entry:", r.text)
    time.sleep(20)     # free tier: minimum 15 s interval
```

6.6 Example 2: Publishing with MQTT (paho-mqtt)

```
import paho.mqtt.client as mqtt, json, time
client = mqtt.Client()
client.connect("broker.hivemq.com", 1883, 60)
client.loop_start()

while True:
    payload = json.dumps({"temp": 28.4, "hum": 61})
    client.publish("iot/lab/pi1/sensors", payload, qos=1)
    time.sleep(10)
```

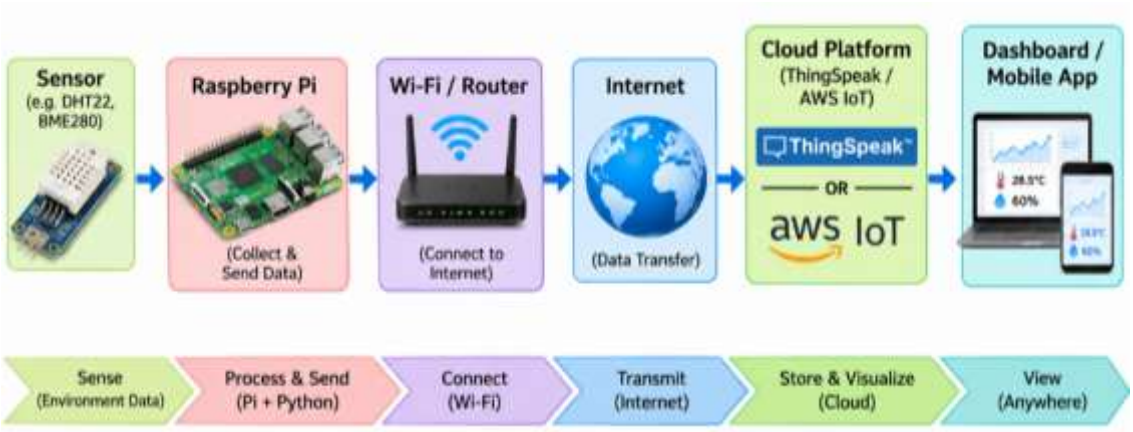


Fig 6.3: Raspberry Pi to Cloud Data Flow (ThingSpeak/AWS IoT)

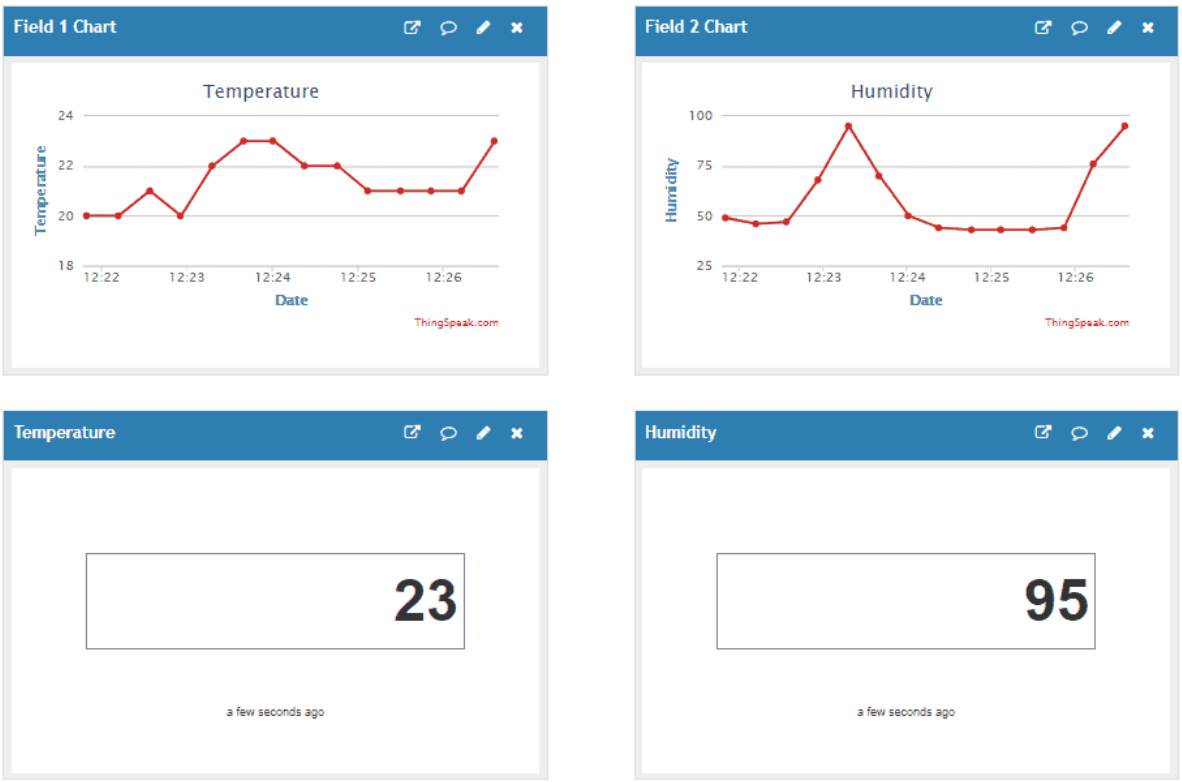


Fig 6.4: ThingSpeak Channel Dashboard (Sample Output)

6.7 Security considerations

- Use TLS/SSL (HTTPS, MQTTS) and device certificates/tokens.
- Never hard-code credentials; use environment variables or config files.
- Change default Pi password, keep OS updated, disable unused services, use firewall.
- Handle network failures: retry logic and local buffering.

7. Implementation of IoT Projects Using Raspberry Pi

7.1 Steps in developing an IoT project

1. **Problem definition and requirements** (what to measure/control, accuracy, cost).
2. **Hardware selection:** Pi model, sensors, actuators, ADC, power supply.
3. **Circuit design and wiring** (level shifting, resistors, isolation).
4. **Software development:** Python scripts for acquisition, processing, control.
5. **Connectivity:** Wi-Fi setup, protocol (MQTT/HTTP), cloud account and API keys.
6. **Dashboard/App** for visualization and control.
7. **Testing, debugging, calibration.**
8. **Deployment:** run script on boot (systemd service or crontab @reboot), enclosure, power backup.

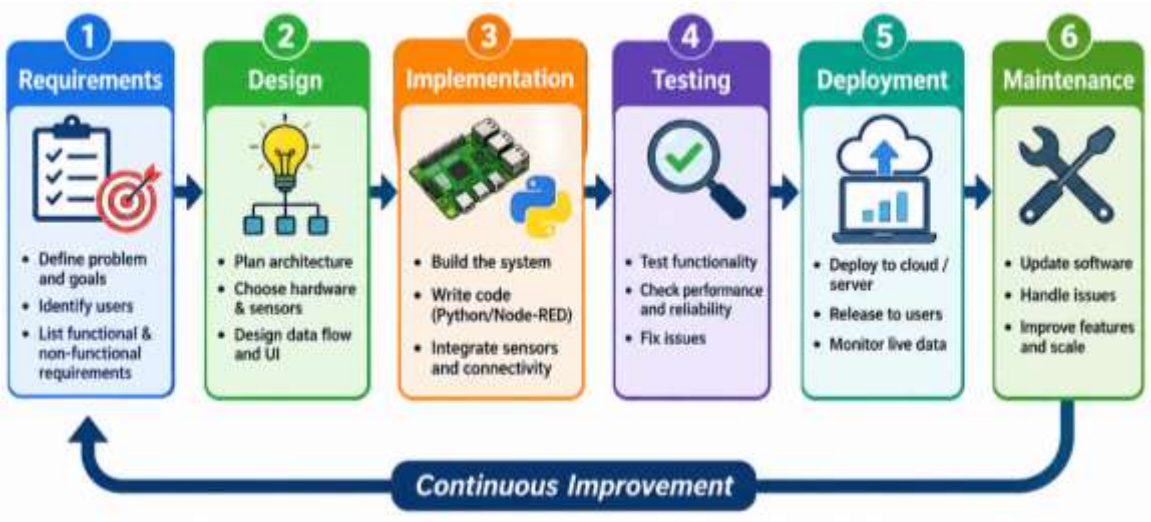


Fig 7.1: IoT Project Development Life Cycle

7.2 Typical system architecture

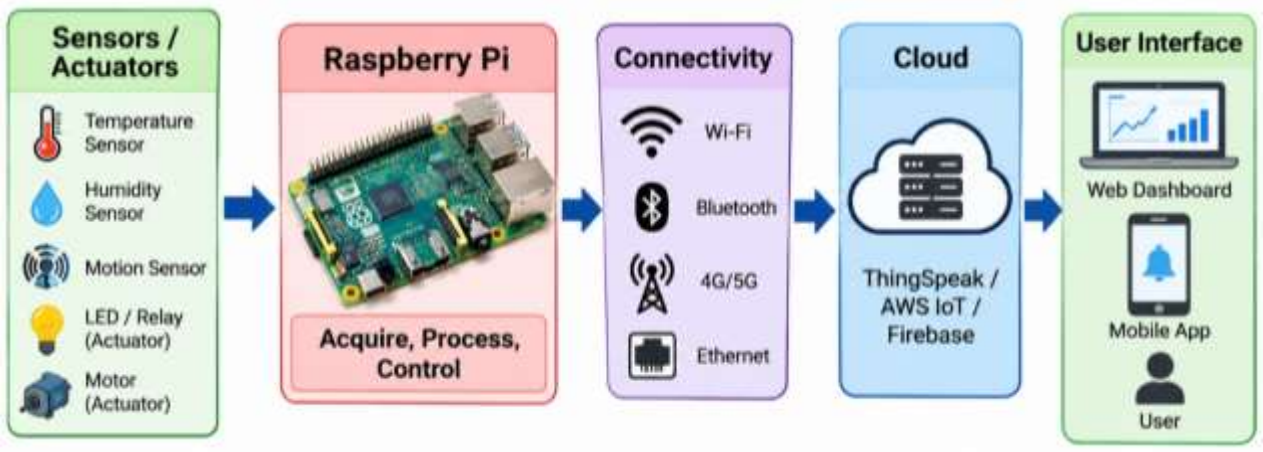


Fig 7.2: Generic Raspberry Pi-Based IoT System Block Diagram

7.3 Auto-start script on boot (systemd)

```
[Unit]
Description=IoT Sensor Service
After=network.target

[Service]
ExecStart=/usr/bin/python3 /home/pi/iot_main.py
Restart=always
User=pi

[Install]
WantedBy=multi-user.target
```

7.4 Mini project: Temperature-controlled fan with cloud logging

- **Hardware:** Raspberry Pi, DHT22, relay module, fan.
- **Logic:** read temperature every 10 s; if > 30 C, fan ON else OFF.
- **Cloud:** upload temperature and fan status to ThingSpeak/MQTT.

- **Output:** live graph on dashboard; optional SMS/email alert on extreme values.

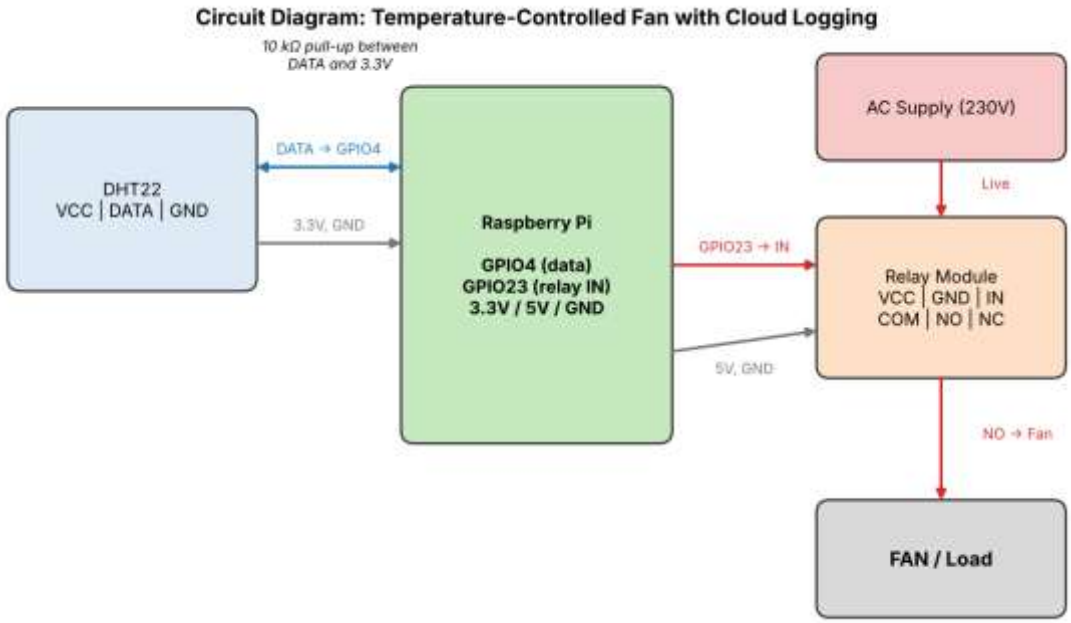


Fig 7.3: Circuit Diagram of Temperature-Controlled Fan with Cloud Logging

```
import time, requests
from gpiozero import OutputDevice
fan = OutputDevice(23, active_high=False)

while True:
    t, h = read_dht()
    if t > 30: fan.on()
    else:      fan.off()
    requests.get(URL, params={'api_key': KEY, 'field1': t, 'field2': int(fan.value)})
    time.sleep(20)
```

8. Case Studies

8.1 Case Study 1: Smart Home Automation

Objective

Monitor and control home appliances, lighting, security and energy use remotely and automatically.

Components

Component	Role
Raspberry Pi	Central home gateway/controller; runs MQTT broker and Home Assistant/Node-RED
Relay modules	Switch lights, fans, appliances
PIR / door-reed sensors	Motion and intrusion detection
DHT22, LDR	Temperature, humidity and ambient light sensing
Gas/smoke sensor (MQ-2)	Safety alerts

Component	Role
Pi Camera	Surveillance and snapshots
Mobile app / voice assistant	User control (Blynk, MQTT dashboard, Alexa)

Working

1. Sensors send data to the Pi; Pi evaluates rules (for example, no motion for 10 min then lights OFF; temperature high then fan ON).
2. User commands from the mobile app reach the Pi through the cloud/MQTT and trigger relays.
3. Intrusion or gas detection triggers a buzzer, camera capture, and push/SMS/email notification.
4. Energy usage and event logs are stored and visualized.

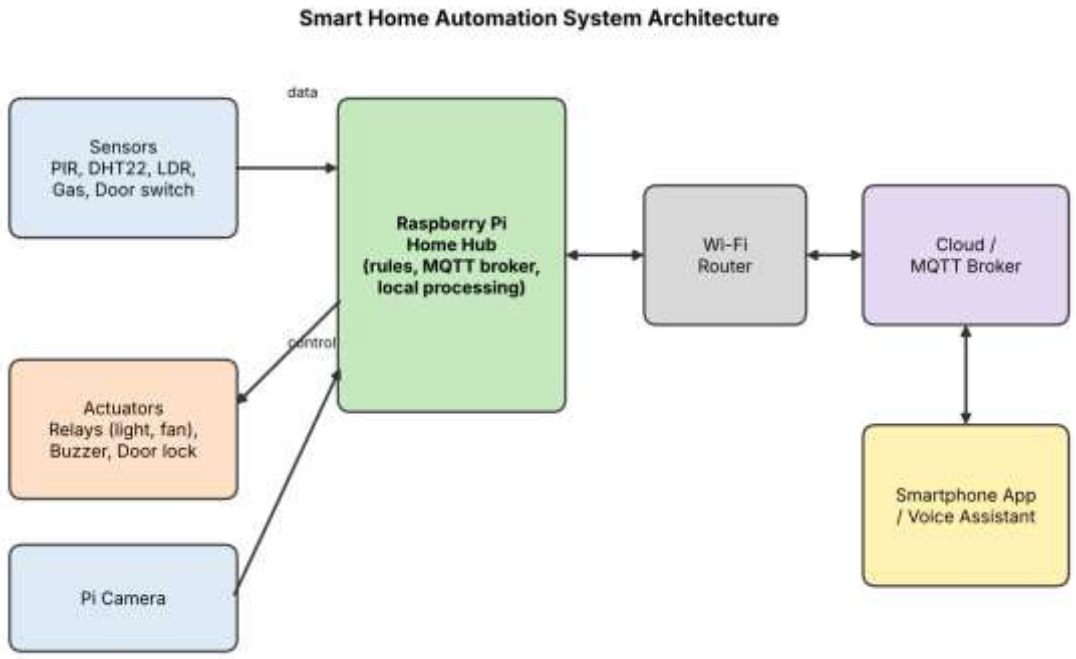


Fig 8.1: Smart Home Automation System Architecture

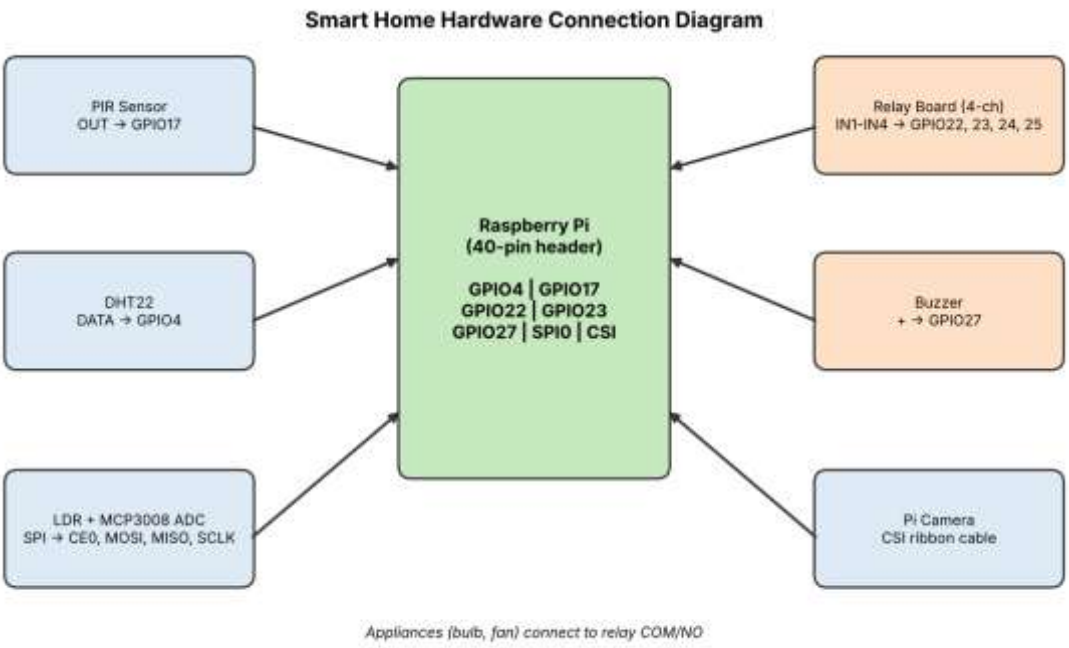


Fig 8.2: Smart Home Hardware Connection Diagram

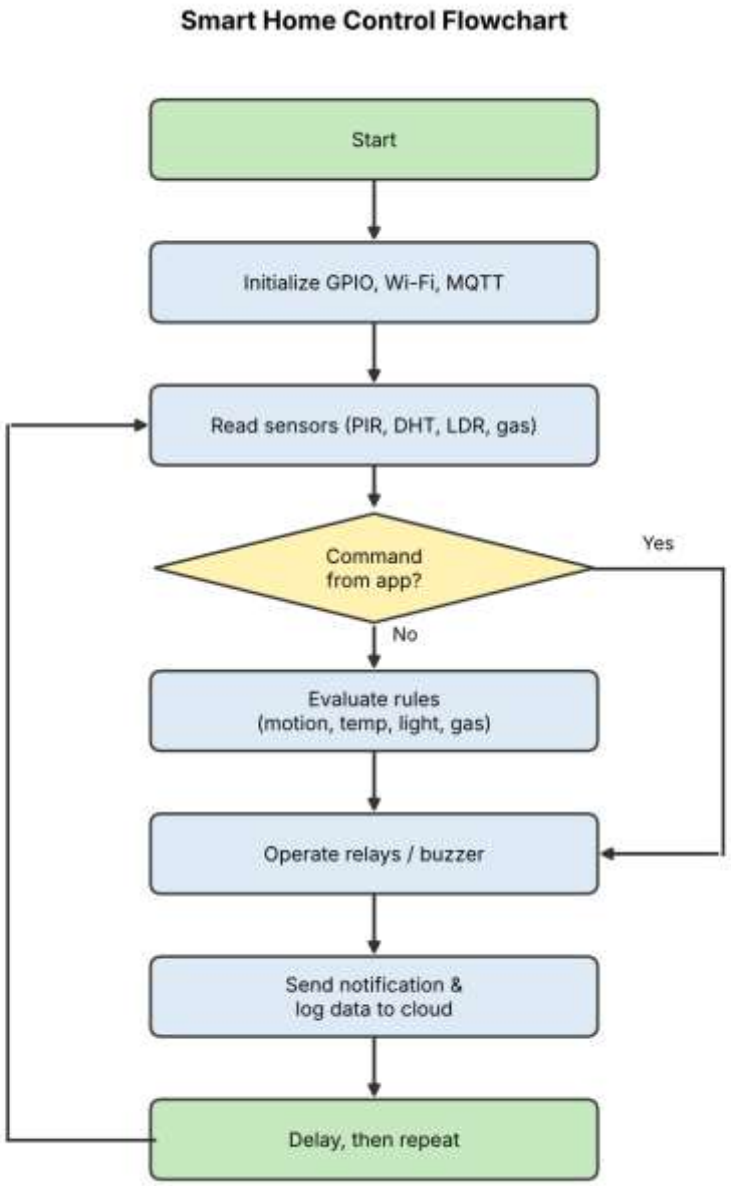


Fig 8.3: Smart Home Control Flowchart

Advantages and challenges

- Advantages: convenience, energy saving, safety, remote access.
- Challenges: security of home network, reliability when internet fails (hence local processing), cost, device interoperability.

8.2 Case Study 2: Healthcare Monitoring

Objective

Continuously monitor patient vital signs and alert doctors or caregivers in case of abnormal readings (remote patient monitoring, elderly care).

Components

Component	Role
Raspberry Pi	Patient-side gateway; processing and transmission
MAX30100/MAX30102 (I2C)	Heart rate and SpO2 (pulse oximetry)
DS18B20 / MLX90614	Body temperature (contact / non-contact)
ECG module AD8232 + MCP3008 (SPI)	ECG signal acquisition
MPU6050 (I2C)	Fall detection for elderly
GSM module (UART)	SMS alerts when internet is unavailable
Display + buzzer	Local indication

Working

1. Sensors acquire vitals at regular intervals via I2C/SPI/1-Wire.
2. Pi filters noise and compares values with safe ranges (for example, SpO2 < 90% or heart rate > 120 bpm).
3. Normal data is uploaded periodically to the cloud (MQTT/HTTP) and shown on a doctor's dashboard; abnormal data triggers an immediate alert by SMS/email/app.
4. Historical data is analysed for trends and predictive insights.

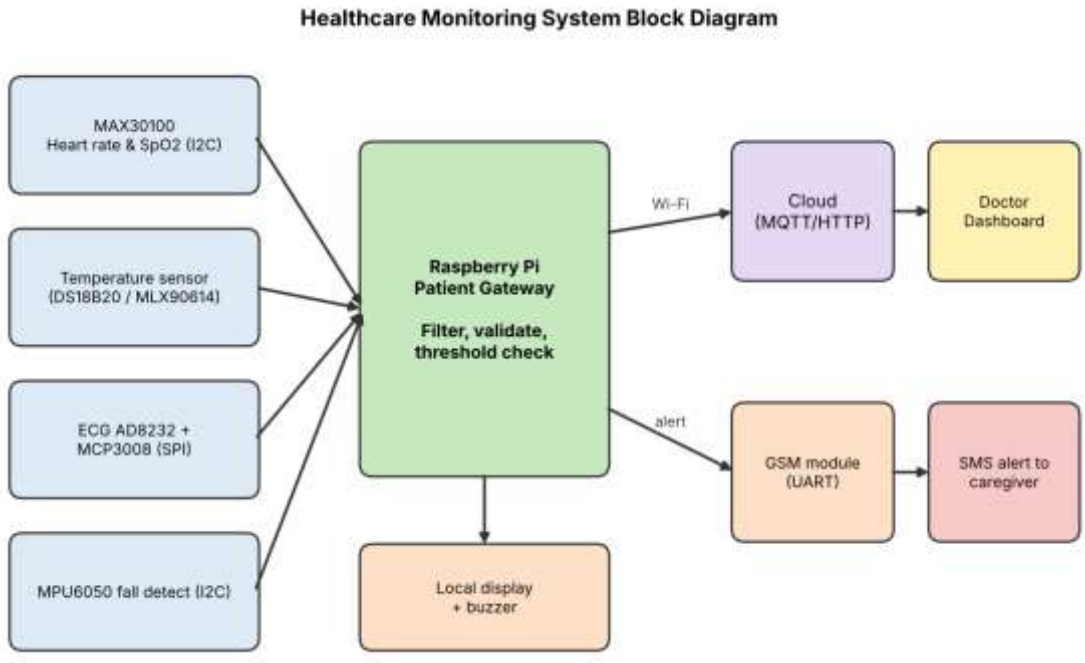


Fig 8.4: Healthcare Monitoring System Block Diagram

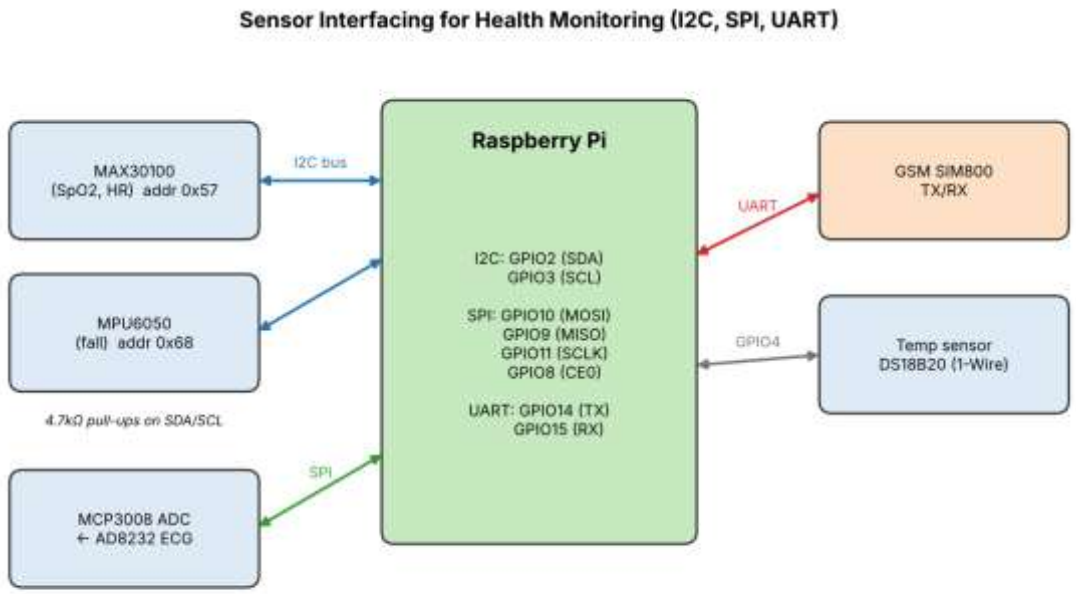


Fig 8.5: Sensor Interfacing for Health Monitoring

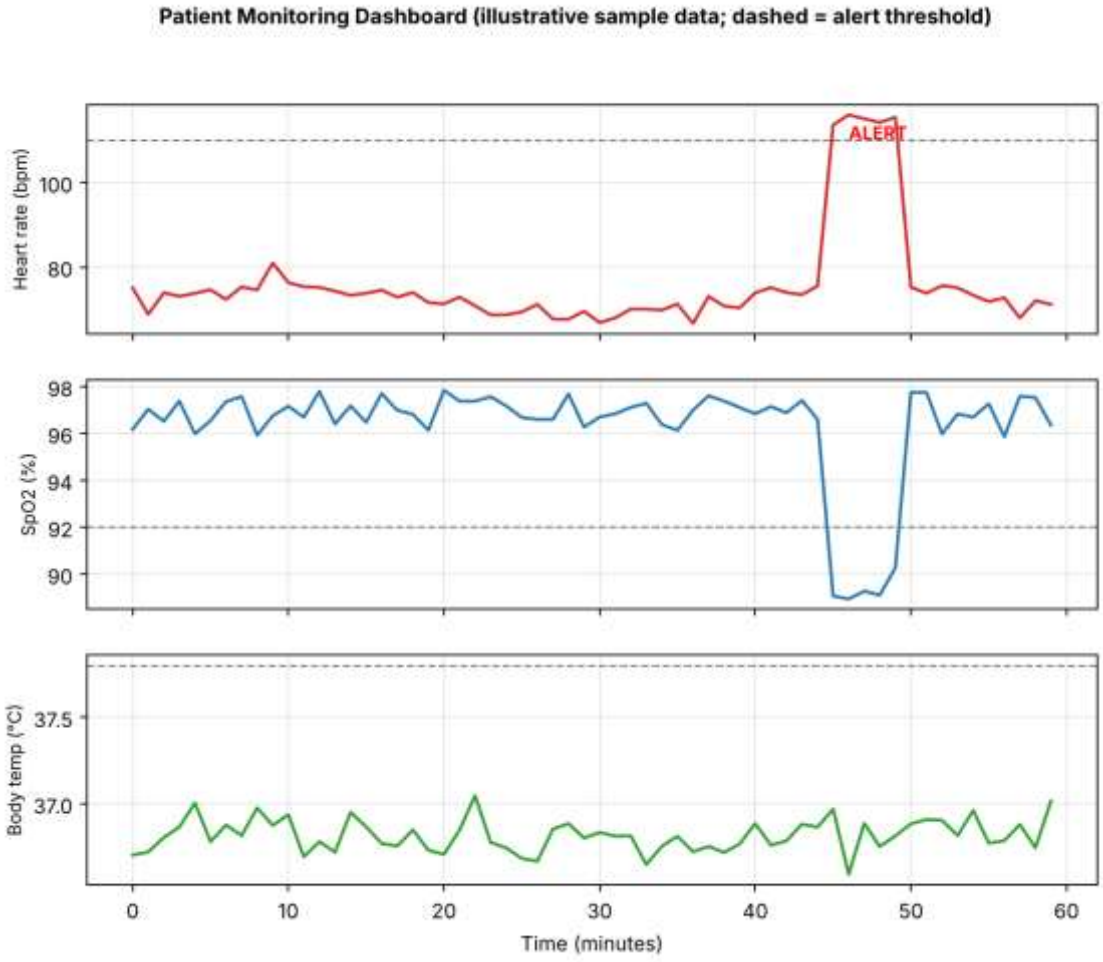


Fig 8.6: Sample Patient Data Dashboard

Advantages and challenges

- Advantages: early detection, reduced hospital visits, continuous monitoring, useful for rural areas.
- Challenges: sensor accuracy and medical certification, patient data privacy and security (encryption, HIPAA-like regulations), battery/power reliability, wearable comfort.

8.3 Case Study 3: Environmental Sensing

Objective

Monitor air quality, weather and other environmental parameters for pollution control, smart cities, and agriculture.

Components

Component	Role
Raspberry Pi	Sensing node/gateway with solar power option
DHT22 / BME280 (I2C)	Temperature, humidity, pressure
MQ-135 / MQ-7 + ADC (SPI)	Air quality, CO2/CO gases
PM2.5/PM10 sensor SDS011 (UART)	Particulate matter
Soil moisture sensor	Agriculture and irrigation
Rain / wind / light sensors	Weather station
Wi-Fi / LoRa / GSM	Data communication

Working

1. Sensors sample environment parameters periodically (for example, every minute).
2. Pi processes data (calibration, averaging, Air Quality Index calculation).
3. Data is published to the cloud (ThingSpeak/AWS IoT) and displayed on web/mobile dashboards or public maps.
4. Alerts are generated when pollution crosses limits; in agriculture, a pump is automatically turned ON when soil moisture is low.

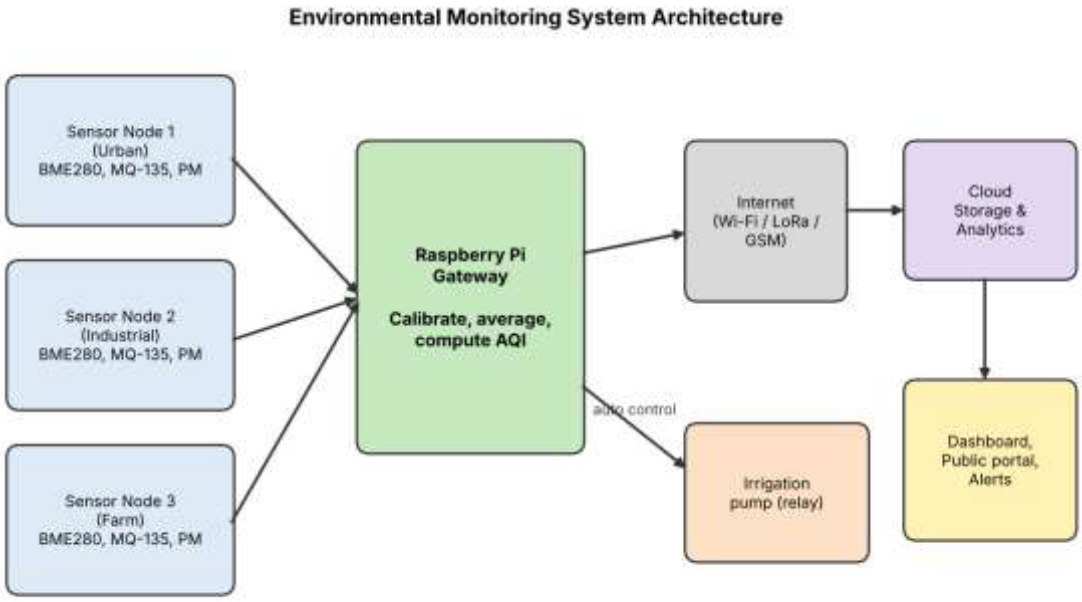


Fig 8.7: Environmental Monitoring System Architecture

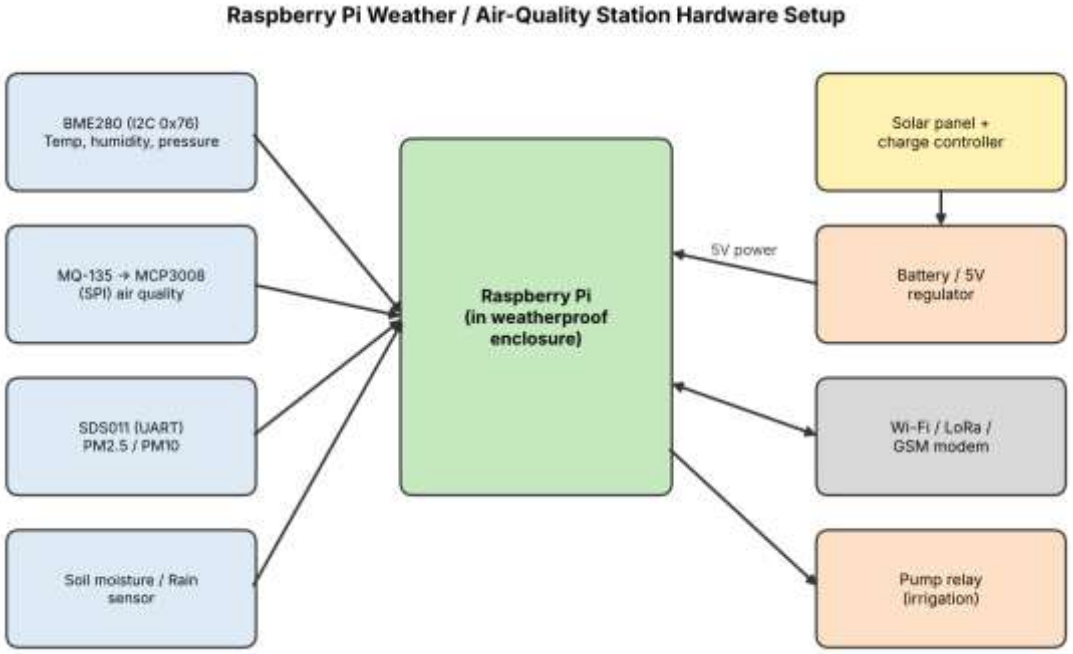


Fig 8.8: Raspberry Pi Weather/Air-Quality Station Hardware Setup

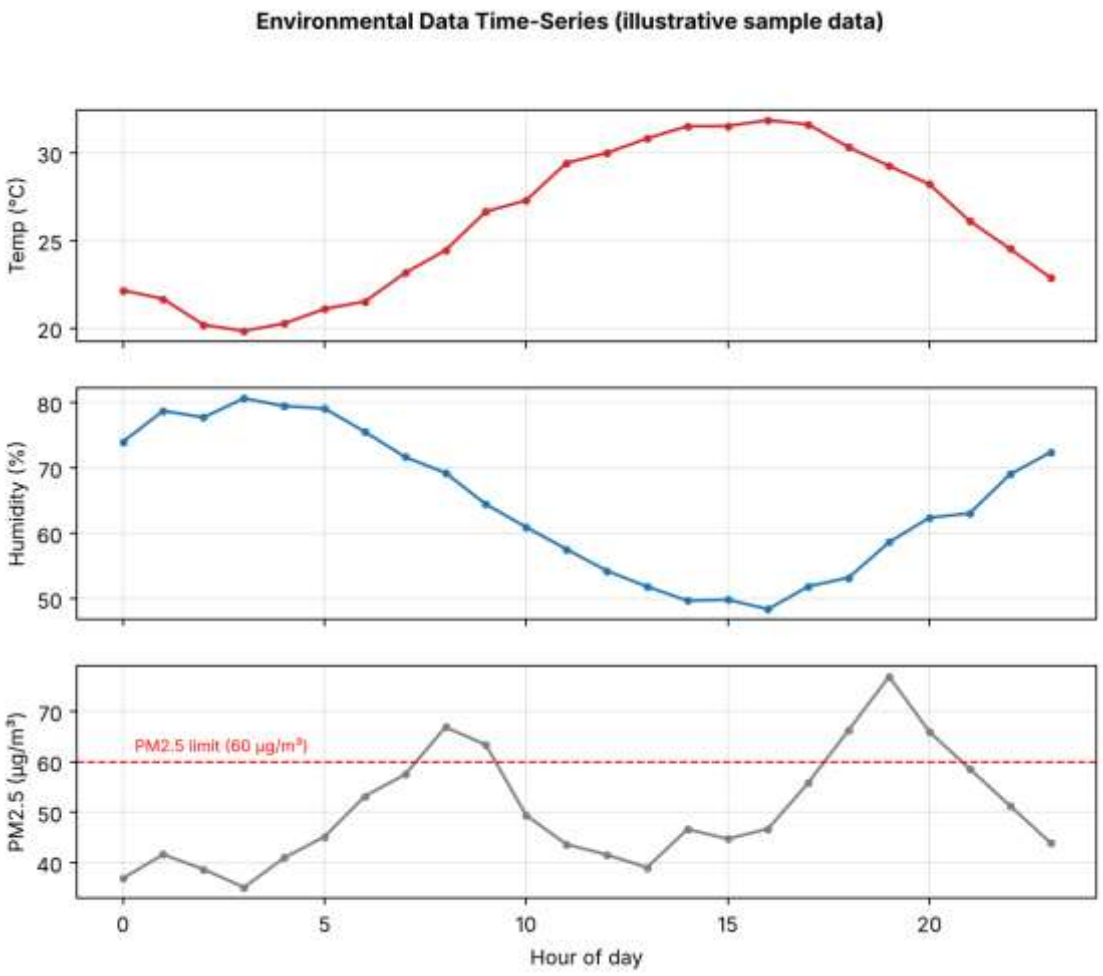


Fig 8.9: Environmental Data Visualization (Time-Series Graphs)

Advantages and challenges

- Advantages: real-time and low-cost monitoring, data-driven policy, early warnings.
- Challenges: sensor calibration and drift, outdoor weatherproofing, power supply in remote areas, network coverage.

8.4 Comparison of the case studies

Aspect	Smart Home	Healthcare	Environmental
Main sensors	PIR, DHT, LDR, MQ-2	MAX30100, ECG, temperature	BME280, MQ-135, PM sensor
Main actuators	Relays, buzzer, locks	Alarm, display	Irrigation pump, alerts
Key requirement	Convenience, security	Accuracy, privacy, reliability	Calibration, durability, coverage
Typical interfaces	GPIO, I2C	I2C, SPI, UART	I2C, SPI, UART
Cloud use	Remote control, logs	Doctor dashboard, alerts	Data analytics, public portal

9. Quick Revision Points

- Python is the preferred IoT language on Raspberry Pi due to simplicity and libraries (gpiozero, RPi.GPIO, spidev, smbus2, pyserial, paho-mqtt).
- Raspberry Pi GPIO works at 3.3 V; no built-in ADC.
- UART: async, 2 wires (TX/RX). SPI: sync, 4 wires, fastest, uses chip select. I2C: sync, 2 wires (SDA/SCL), address-based, needs pull-ups.
- DAQ chain: sensor, signal conditioning, ADC, processing, storage.
- Edge processing reduces latency and bandwidth.
- MQTT is a lightweight publish/subscribe protocol using a broker; HTTP is request/response.
- Cloud platforms: ThingSpeak, AWS IoT Core, Azure IoT Hub, Firebase, Adafruit IO.
- Case studies: smart home (control and security), healthcare (vitals and alerts), environment (air/weather monitoring).